RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

VisuaLearn AI: An Adaptive Multi-Modal Intelligent Learning Platform

A Major Project Report (IS481P)

Submitted by,

Jeevan Raj S B	**1RV22IS026**
R A Nithin Nandana	**1RV22IS047**
Siddesh K R	**1RV22IS064**
Amulya S	**1RV23IS400**

Under the guidance of

Prof. Chethana R	**Dr. Kavitha S N**
Assistant Professor	Associate Professor
Dept. of ISE	Dept. of ISE
RV College of Engineering	RV College of Engineering

In partial fulfillment of the requirements for the degree of

Bachelor of Engineering in

Information Science and Engineering

2025-26

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Information Science and Engineering

CERTIFICATE

Certified that the major project (IS481P) work titled ***VisuaLearn AI: An Adaptive Multi-Modal Intelligent Learning Platform*** is carried out by **Jeevan Raj S B** (**1RV22IS026**), **R A Nithin Nandana** (**1RV22IS047**), **Siddesh K R** (**1RV22IS064**) and **Amulya S** (**1RV23IS400**) who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Information Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Internal Guide	External Guide	Head of the Department	Principal
Prof. Chethana R	Dr. Kavitha S N	Dr.G.S. Mamatha	Dr. K.N. Subramanya

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

We, **Jeevan Raj S B**, **R A Nithin Nandana**, **Siddesh K R** and **Amulya S** students of eighth semester B.E., Department of Information Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**VisuaLearn AI: An Adaptive Multi-Modal Intelligent Learning Platform**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Information Science and Engineering** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

	Name	Signature
1.	Jeevan Raj S B(1RV22IS026)	
2.	R A Nithin Nandana(1RV22IS047)	
3.	Siddesh K R(1RV22IS064)	
4.	Amulya S(1RV23IS400)	

ACKNOWLEDGEMENTS

We are deeply indebted to our internal guide, **Prof. Chethana R**, Assistant Professor, Department of ISE, RVCE, for the wholehearted support, suggestions, and valuable advice throughout our project work and for guiding us in the preparation of this report.

We express our sincere gratitude to our external guide, **Dr. Kavitha S N**, Associate Professor, Dept. of ISE, RV College of Engineering, for her guidance, technical inputs, and encouragement during the development of VisuaLearn AI.

Our sincere thanks to the project coordinator, **Dr. Vanishree K**, Associate Professor, Department of ISE, for the timely instructions and support in coordinating the project.

We are grateful to **Dr.G.S. Mamatha**, Professor and Head, Department of Information Science and Engineering, RVCE, for the support and encouragement.

We express sincere gratitude to **Dr. K.S Geetha**, Vice-Principal, RVCE, and **Dr. K.N. Subramanya**, Principal, RVCE, for providing the academic environment and institutional support required to complete this project successfully.

We thank all the teaching staff and technical staff of the Department of Information Science and Engineering, RVCE, for their help. Lastly, we thank our family members and friends who provided constant support throughout the project work.

ABSTRACT

VisuaLearn is an AI-powered educational platform designed to support interactive, personalized, and visual learning experiences. Many existing online learning systems provide static notes or generic chatbot responses that may not align with a learner's current knowledge level, preferred explanation style, or need for visual understanding. VisuaLearn addresses these limitations by integrating conversational AI, dynamically generated visual components, personalized learning plans, user profiles, feedback mechanisms, and persistent learning history into a unified web application.

The objective of this project is to design and implement a full-stack learning platform capable of authenticating users, storing learning sessions, streaming AI-generated explanations, generating interactive learning assets, and adapting responses based on learner context. The frontend is developed using React, Vite, Tailwind CSS, and reusable hooks and components, while the backend is implemented using Node.js and Express. Supabase is utilized for authentication, PostgreSQL-based data persistence, and storage services. Optional Azure OpenAI services are integrated to support text-to-speech, image generation, and real-time voice interactions.

The system includes modules for conversational learning, learning plans, adaptive content generation, document-based learning, user feedback, personas, simulations, 3D visualizations, video generation, and learning-state tracking. Server-Sent Events (SSE) are used to stream AI responses to the client in real time, while backend middleware ensures secure route protection through authentication, ownership validation, rate limiting, logging, and error handling. Generated widgets and visual elements undergo safety validation before rendering.

A structured user-evaluation plan for 30 participants was prepared to assess usability, learning effectiveness, and user experience across different learning modules. The planned evaluation focuses on how learners interact with visual explanations, simulations, revision artifacts, and personalized responses compared with static learning approaches. This framing keeps the report ready for real participant data without presenting uncollected results as completed findings.

The implemented system demonstrates how artificial intelligence can be effectively integrated with modern web application engineering to create an engaging and adaptive learning environment. This report presents the theoretical background, system architecture, implementation details, evaluation plan, and supporting materials required to understand, assess, and extend the VisuaLearn platform.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction

Chapter 1

Introduction

VisuaLearn is an AI-assisted educational platform designed to make abstract concepts easier to understand through conversational explanation, visual widgets, adaptive feedback, simulations, revision artifacts, and persistent learning history. The project addresses a common limitation in digital learning systems: students often receive text-heavy explanations that do not adapt to their level, preferred learning mode, or current confusion. The system combines a React frontend, an Express backend, Supabase authentication and storage, and large language model based tutoring to create a more interactive learning environment.

1.1 Background

Digital learning platforms have become an important part of modern education. Students use online resources, recorded lectures, search engines, discussion forums, and learning applications to clarify doubts and revise concepts. These tools provide access to information, but they often do not provide a complete learning experience. A learner may receive an explanation but still need diagrams, examples, practice questions, revision cards, or a step-by-step simulation to understand the topic fully.

Recent work in online and blended learning shows that learner engagement remains a major challenge even when digital access is available [1], [2]. Research in multimedia learning also shows that coordinated verbal and visual representations can improve conceptual understanding compared with isolated text [3]. Simulation-based learning studies further report that interactive representations can support understanding of procedural and complex skills [4]. These findings indicate that an effective digital learning system should not only answer questions, but should also support visual explanation, interaction, revision, and feedback.

Technical subjects such as data structures, algorithms, computer networks, operating systems, mathematics, and engineering science require multiple representations. For example, students learning sorting algorithms may understand the definition but struggle to visualize comparisons and swaps occurring internally. Similarly, recursion, graph traversal, memory allocation, and CPU scheduling become difficult when represented only through static text. VisuaLearn is designed to respond to this need by combining explanation, visualization, practice, simulation, and saved revision material in one web-based learning workspace.

1.2 Literature Review

The design of VisuaLearn is influenced by research in multimedia learning, cognitive load theory, intelligent tutoring systems, adaptive learning, and AI-assisted education. Mayer's multimedia learning theory explains that learners understand better when verbal and visual information are coordinated meaningfully rather than presented as disconnected material [3]. Sweller's cognitive load theory explains that learning quality depends on the mental effort required to process new information [5]. These theories support the use of concise explanations, visual widgets, simulations, and progressive learning artifacts.

Bloom's work on one-to-one tutoring showed the strong potential of personalized instruction [6]. Later research on intelligent tutoring systems showed that software-based tutors can improve learning when they model learner progress and provide timely feedback [7], [8]. Recent studies on large language models in education highlight their ability to generate natural-language explanations, while also warning about reliability, hallucination, feedback quality, and the need for careful pedagogical integration [9].

Existing systems often focus on one layer of learning support, such as chat-based explanation, quizzes, visualization, or course delivery. VisuaLearn attempts to combine these layers into one adaptive workflow. It uses learner context, generated visual elements, document-assisted learning, structured revision, simulations, feedback, and persistent history so that learners can move from explanation to practice and revision without switching tools.

1.3 Motivation

The motivation for VisuaLearn comes from the difficulty students face when learning complex topics from static material. A learner may understand a concept faster when it is shown as a diagram, animated process, chart, analogy, code walkthrough, or quiz. However, most platforms require users to search separately for each form of support. This breaks learning continuity and increases the effort required for revision.

For example, a student learning bubble sort may memorize that adjacent elements are compared and swapped, but may not understand how the largest element moves to the end after each pass. A student learning recursion may understand the code but not the call stack. A student learning graph traversal may know the definition of BFS but not how the queue changes at each step. These cases show why visual and step-by-step learning support is important.

The project is also motivated by the need for personalization. Two users may ask the same question but require different responses. A beginner may need a simple analogy, while an advanced user may need formal reasoning or implementation details. A confused learner may need guided steps, while a learner preparing for assessment may need quizzes and flashcards. VisuaLearn records user preferences and interaction signals so that future responses can be adapted.

1.4 Problem statement

Existing learning systems generally provide isolated learning features and rarely combine adaptive explanations, visual simulations, revision artifacts, document-based context, and learner history into a unified workflow. This results in fragmented learning experiences, reduced engagement, and limited support for concept retention. The problem is to design and implement a web-based AI learning platform that can deliver personalized, interactive, and persistent learning support using conversational AI, generated visual assets, user profiles, and feedback-driven adaptation.

1.5 Objectives

The objectives of the project are:

1. To design a full-stack agentic AI learning platform that supports authenticated users, persistent conversations, learning profiles, and adaptive tutoring.

2. To implement an agent-based learning pipeline for streaming explanations, visual widgets, learning plans, document-based learning, simulations, 3D visualizations, video generation, and optional voice interaction.

3. To provide a feedback and personalization mechanism that helps AI agents improve the relevance of explanations based on user profile, behavior, persona, and learning state.

4. To evaluate system usability and learning effectiveness through participant testing, feedback analysis, and interaction observations.

1.6 Scope of Project

The scope of this project is limited to a web-based AI learning environment that supports educational queries, generated explanations, visual artifacts, simulations, adaptive learning, revision resources, and persistent learning history. The system supports authenticated learners, stores conversations and learning resources, and provides optional voice interaction where provider credentials and browser permissions are available.

The platform is intended for educational domains where visual or procedural explanation is meaningful, such as computer science, engineering, mathematics, and related technical subjects. It is not intended to replace teachers, certified examinations, or formal grading systems. Generated content is treated as learning support and should be verified for high-stakes academic use.

1.7 Brief Methodology of the project

The project follows an iterative software engineering methodology. First, the learning problem and expected user workflows were identified. Next, the system was divided into frontend, backend, database, and AI-service responsibilities. The frontend was implemented as a React application with reusable components and hooks, while the backend was implemented as an Express API with route modules, middleware, agents, services, and database utilities. Supabase was selected for authentication and persistence, and AI providers were integrated through backend services to avoid exposing secret keys in the browser.

The learning workflow begins when the learner submits a query. The system processes the query, retrieves relevant conversation or document context, selects a response strategy, generates the required learning artifact, validates the output, stores the resource, and displays it in the learning workspace. Learner interactions such as quiz attempts, completion, feedback, response time, and resource usage are recorded for future personalization. The completed system is prepared for evaluation using participant feedback, usability measures, and interaction observations.

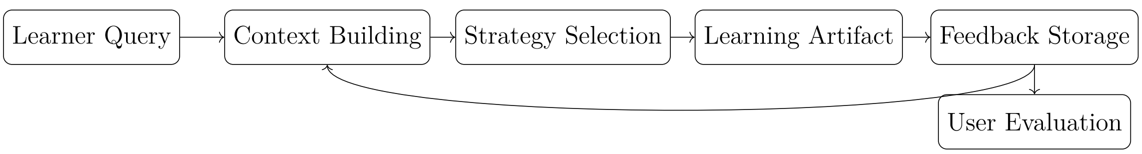

Figure 1.1: High-level methodology and adaptive learning workflow

1.8 Assumptions made / Constraints of the project

The project assumes that users have internet access, a modern browser, and valid authentication through Supabase. The backend requires configured environment variables for Supabase and AI-provider services. Optional features such as text-to-speech, image generation, and real-time voice depend on Azure OpenAI configuration. Since generated AI content can contain mistakes, the system includes feedback and validation mechanisms, but final academic correctness still depends on validation by the learner or teacher.

The major constraints are API cost, external service availability, model latency, browser performance for 3D rendering, and the quality of generated educational assets. The platform is valid as an AI-assisted learning aid, not as a certified assessment or grading system. It is intended to support explanation, visualization, revision, and self-study.

The implemented platform is prepared for evaluation using approximately 30 users interacting with learning modules such as simulations, visual explanations, document-based learning, and revision artifacts. Feedback collected from such participants can be used to analyze usability and learning effectiveness.

Chapter 2

Fundamentals and Literature Survey

Chapter 2

Fundamentals and Literature Survey

The development of VisuaLearn is based on concepts from intelligent tutoring systems, adaptive learning, learner modelling, educational visualization, simulation-based learning, structured revision, and reliable full-stack software design. This chapter separates the theoretical fundamentals from the literature survey so that the project can be understood as both a software system and a research-motivated educational tool.

2.1 Fundamentals

2.1.1 Intelligent Tutoring Systems

An Intelligent Tutoring System (ITS) is a software system that provides instruction, feedback, and support to learners. Such systems usually contain a domain model, a learner model, a tutoring model, and an interface model. The domain model represents subject knowledge. The learner model stores the student's current understanding. The tutoring model decides the instructional action. The interface model presents the response to the learner.

VisuaLearn follows the same educational principle but implements it in a modern web-based form. The domain content is generated dynamically for different topics. The learner model is represented using conversation history, resource history, quiz results, engagement signals, and adaptive context. The tutoring model selects teaching strategies such as visual explanation, guided steps, quiz mode, Socratic questioning, or remediation.

2.1.2 Adaptive Learning

Adaptive learning modifies instruction based on learner needs. A learner who answers correctly may be ready for a harder example, while a learner who repeatedly fails may need remediation. A learner who spends more time on a topic may need a slower explanation, and a learner who quickly completes content may prefer a concise response.

In this project, adaptation is performed at the response level. The system does not only decide what topic should come next; it also decides how the current topic should be taught. The adaptive layer uses context features such as cognitive state, engagement level, topic status, performance trend, confidence, topic difficulty, previous failures, and response time. These features help the system select a suitable teaching mode.

2.1.3 Educational Visualization and Simulation

Educational visualization represents abstract ideas using diagrams, charts, animations, and interactive components. It is useful when learners need to understand structure, movement, sequence, or relationships. In computer science, visual learning helps explain algorithms, data structures, recursion, memory, and state transitions.

Simulation-based learning is useful for processes that can be represented as a sequence of states. Sorting algorithms can be shown through comparisons and swaps. Graph traversal can be shown through visited nodes and queues. CPU scheduling can be shown through timelines and process queues. This supports active learning because the learner observes the process rather than only reading about it.

2.1.4 Structured Revision

Learning does not end with understanding an explanation. Students need revision and practice to retain concepts. Flashcards support recall. Quizzes support self-assessment. Mind maps support conceptual organization. Examples support transfer of learning to new situations. VisuaLearn generates these revision artifacts from the learning session so that a single query can lead to notes, examples, flashcards, quizzes, simulations, and saved resources.

2.2 Literature Survey

Bloom [6] showed that individualized tutoring can produce substantially stronger learning outcomes than conventional classroom-only instruction. The limitation of human tutoring is scalability: one-to-one tutoring is difficult to provide continuously to every learner. This motivates software systems that approximate personalized guidance.

VanLehn [7] reviewed the relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. The study showed that tutoring effectiveness depends on the granularity and quality of interaction. This is relevant to VisuaLearn because the platform attempts response-level adaptation rather than only course-level sequencing.

Ma et al. [8] conducted a meta-analysis of intelligent tutoring systems and learning outcomes. Their work supports the claim that ITS-style personalization can improve learning when learner state is modeled. However, many ITS implementations are domainspecific and do not naturally support open-ended learner questions, generated visual artifacts, or multimodal revision resources.

Mayer [3] and Sweller [5] provide the theoretical foundation for combining verbal and visual explanation while controlling cognitive load. Their work is directly relevant to visual explanations, simulations, and step-by-step breakdowns. The limitation is that these theories do not by themselves define a deployable AI learning platform.

Kasneci et al. [9] discuss opportunities and challenges of large language models in education. LLMs can generate flexible explanations and respond to natural-language questions, but educational systems must handle hallucination, over-reliance, feedback quality, and learner safety. VisuaLearn addresses this through backend orchestration, validation, feedback, and persistent learning history.

Recent AI-in-education reviews also emphasize that AI systems should support personalization, feedback, and responsible deployment rather than act as isolated answer generators [10]. This supports the project goal of building an integrated learning platform with authentication, storage, adaptive routing, and safe rendering.

2.3 Literature Comparison Table

Table 2.1: Comparison of Existing Work with VisuaLearn

Existing Work	Adaptive	Visual	Sim.	Revision	Personal.
Bloom tutoring model [6]	Yes	No	No	Partial	High
Traditional ITS [7], [8]	Yes	Partial	Partial	Partial	Yes
Adaptive hypermedia [11]	Yes	Partial	No	Partial	Yes
Multimedia learning systems [3]	No	Yes	Partial	No	No
LLM-based tutoring [9]	Partial	Partial	No	Partial	Partial
VisuaLearn AI	Yes	Yes	Yes	Yes	Yes

2.4 Research Gap

Existing systems commonly focus on isolated educational tasks such as tutoring, quizzes, visualization, or content delivery. Very few systems integrate adaptive teaching, simulations, generated revision material, multimodal interaction, document-based learning, and learner-context modeling within a single environment. Additionally, limited attention has been given to safe rendering and persistence of dynamically generated educational artifacts.

The major gaps identified from the literature are:

- Many systems provide explanations but do not generate a complete set of learning artifacts.
- Visualization and simulation are often separate tools rather than part of the tutoring flow.
- Revision content is usually manually created by the learner.
- Learner context is not always used to decide the teaching format for each response.
- Safe rendering of generated interactive learning content is not consistently addressed.
- LLM-based learning tools often lack persistent learning history, structured evaluation, and adaptive feedback loops.

2.5 Proposed Solution

VisuaLearn addresses the identified research gaps through a unified adaptive learning architecture that combines conversational AI, visualization, simulations, revision generation, learner modeling, document-based learning, feedback, and persistent learning history. Unlike systems that provide only a chatbot response or a separate visualization tool, the proposed system supports multiple teaching strategies and dynamically selects the most suitable learning mode based on learner context.

The platform also treats generated learning content as a managed artifact. Responses, visual widgets, simulations, quizzes, flashcards, and learning resources are stored with the learner session. This supports continuity, revision, and future adaptation. Safety mechanisms such as authentication, route protection, validation, fallback handling, and logging are included so that generated content can be used more reliably.

2.6 Summary

The literature supports the value of personalized instruction, visual learning, adaptive feedback, and structured revision. However, the reviewed work also shows that many systems treat these capabilities separately. VisuaLearn is positioned as an integrated AI learning platform that brings these features into a single workflow and prepares the system for usability and learning-effectiveness evaluation.

Chapter 3

Analysis and Design

Chapter 3

Analysis and Design

VisuaLearn is designed as a layered, modular, and agent-driven learning platform. The system separates presentation, routing, context building, adaptive decision-making, learning artifact generation, validation, storage, and feedback. This chapter presents the requirements, technology stack, system architecture, diagrams, data model, API flow, security design, and design rationale.

3.1 Requirement Analysis

The functional requirements identified for the project are:

1. The system shall allow users to sign up, log in, and access protected learning features.
2. The system shall provide a chat interface for educational questions and streamed AI responses.
3. The system shall save conversations, messages, learning resources, and generated artifacts.
4. The system shall generate explanations, quizzes, flashcards, mind maps, simulations, image outputs, video outputs, and 3D visualizations where suitable.
5. The system shall use agentic AI modules for context building, adaptive strategy selection, content generation, validation, and feedback handling.
6. The system shall support document-based learning through retrieval over uploaded resources.
7. The system shall collect feedback and interaction signals for personalization.
8. The system shall optionally support voice input and speech output.

The non-functional requirements are:

1. **Scalability:** The backend should support modular route expansion and independent feature services.

2. **Reliability:** Failed AI or visualization responses should degrade gracefully with fallback output.

3. **Performance:** Streaming responses should reduce perceived latency, and expensive generation routes should be rate limited.

4. **Security:** User data and provider keys should be protected through authentication, authorization, backend-only secrets, and safe rendering.

5. **Maintainability:** Frontend components, backend routes, services, and agents should remain separated by responsibility.

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, Tailwind CSS	Builds the single-page interface, chat workspace, learning tabs, and responsive UI.
Backend	Node.js, Express	Handles API routing, middleware, streaming, validation, orchestration, and service integration.
Database and Auth	Supabase, PostgreSQL	Provides authentication, user profiles, conversations, messages, feedback, and resource persistence.
AI Services	LLM providers, Azure OpenAI optional services	Supports text generation, speech, image generation, video generation hooks, and real-time interaction where configured.
Learning APIs	Chat, simulation, visualization, voice, feedback, documents	Exposes modular endpoints for the main learning workflows.
Rendering	React components, safe frames, declarative viewers	Displays generated learning artifacts while limiting unsafe output.

Table 3.1: Technology stack used in VisuaLearn

3.3 System Architecture

The system architecture follows the layered structure shown in Figure 3.1. The presentation layer sends requests to the application layer, the intelligence layer builds context and chooses a strategy, core and multimodal modules generate learning artifacts, support modules validate and format output, and the data layer stores profile, history, logs, and knowledge resources.

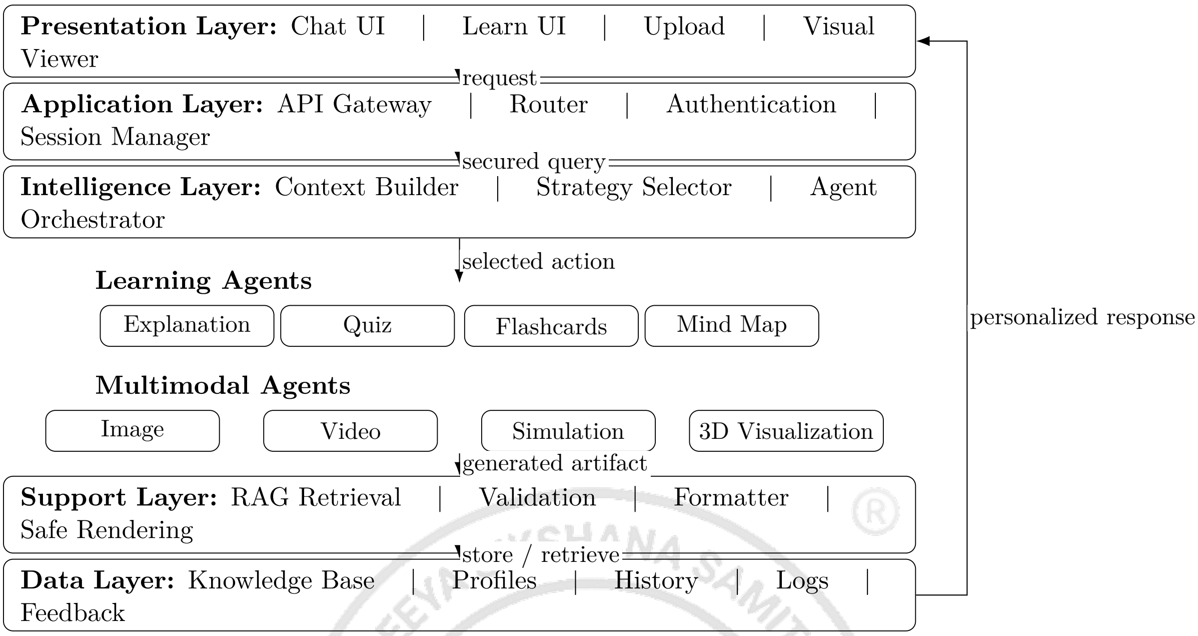

Figure 3.1: Detailed system architecture of VisuaLearn

3.4 Use Case Diagram

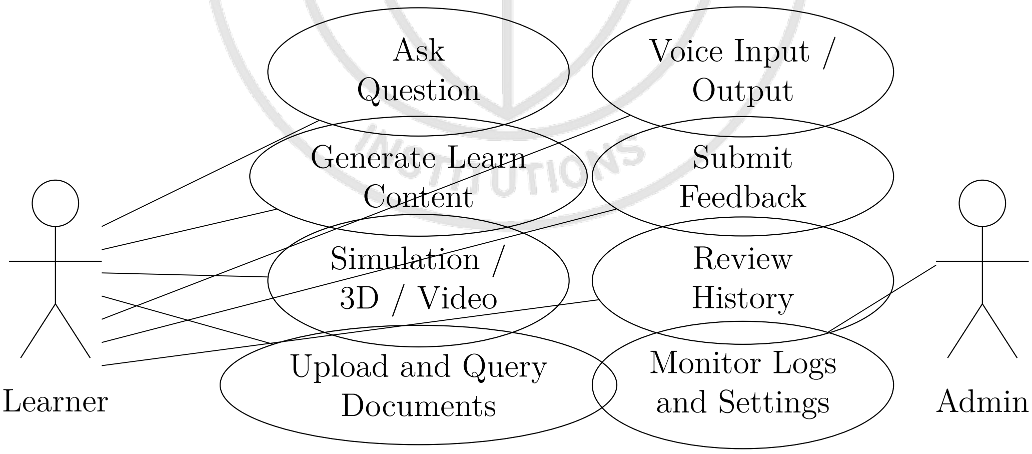

Figure 3.2: Use case diagram of VisuaLearn

3.5 Sequence Diagram for Learning Flow

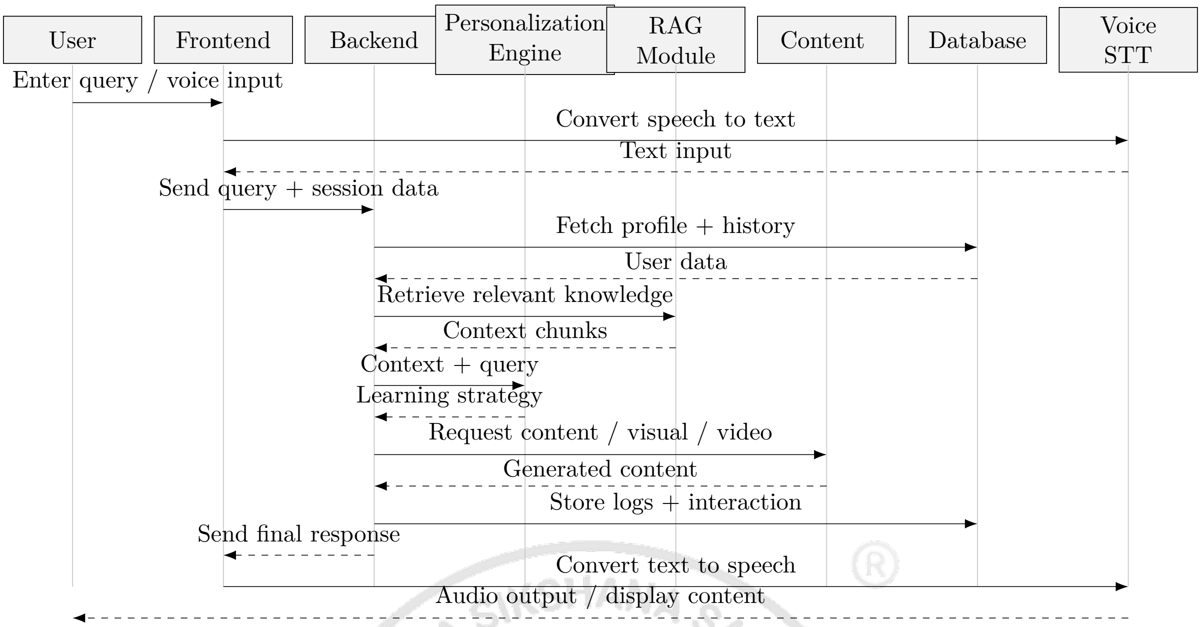

Figure 3.3: Sequence diagram for adaptive learning flow

3.6 Data Design

The core database design supports authenticated learning sessions, generated artifacts, feedback, and adaptive decisions. Table 3.2 improves the main entity list by including ownership and relationships.

Table / Entity	Stored Data	Relationship / Purpose
user_profiles	Learning style, language, comprehension level, pace preference, profile metadata	One profile belongs to one authenticated learner and guides personalization.
conversations	Conversation title, owner, timestamps, summary	A user can have many learning sessions.
messages	User, assistant, and system messages with metadata	Messages belong to conversations and preserve learning history.
learning_resources	Notes, quizzes, flashcards, mind maps, simulations, visual references	Generated artifacts are linked to conversations and reused during revision.
documents	Uploaded files, extracted chunks, embeddings, source metadata	Supports RAG retrieval for document-based learning.
feedback	Ratings, corrections, completion, quiz results, usefulness signals	Feedback is connected to messages/resources and informs future adaptation.
agent_decisions	Context vector, selected strategy, confidence, reward data	Stores adaptive strategy selection for analysis and improvement.
personas	Tutor style, tone, explanation preference	Personas influence response generation and user experience.

Table 3.2: Important data entities in VisuaLearn

3.7 Database Schema / ER Diagram

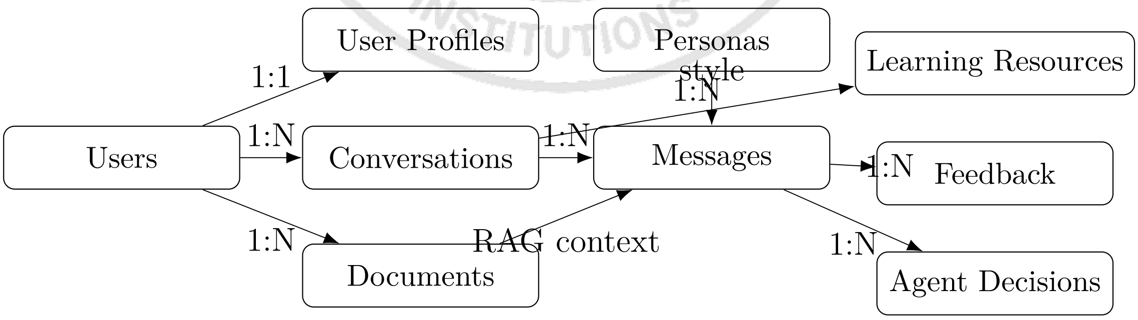

Figure 3.4: Database schema and entity relationship diagram

3.8 Detailed Data Flow Diagram

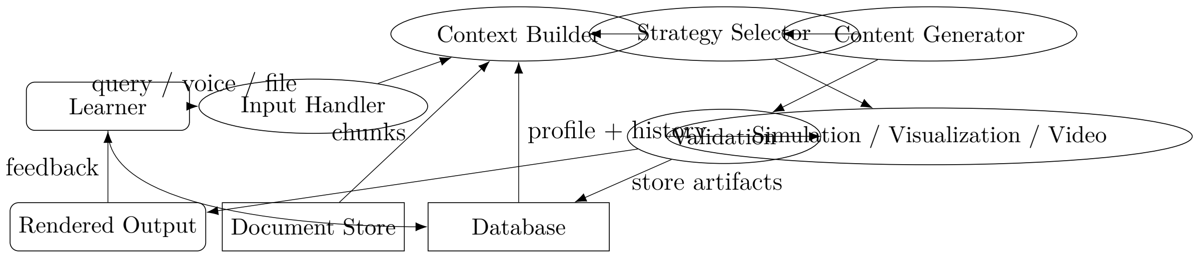

Figure 3.5: Detailed data flow diagram

3.9 API Flow Description

Route Group	**Main Responsibility**	**Flow**
Chat	Conversational learning and streaming response	Receives query, loads session context, invokes strategy selection, streams generated response, and stores messages.
Simulation	Procedural visual learning	Detects simulation suitability, generates declarative steps/specification, validates output, and returns a renderable simulation payload.
Visualization	Image, video, and 3D learning support	Builds visual-generation request, validates generated artifact or spec, and sends it to the safe viewer.
Voice	Speech input and output	Converts speech to text, sends text through learning flow, and converts the final response to speech when enabled.
Feedback	Learning signal collection	Stores usefulness, quiz correctness, completion, and correction signals for future personalization.
Documents	Document-assisted learning	Uploads files, extracts text, chunks content, stores metadata, and retrieves relevant context for later queries.

Table 3.3: Major API route flows

3.10 Module Dependency Diagram

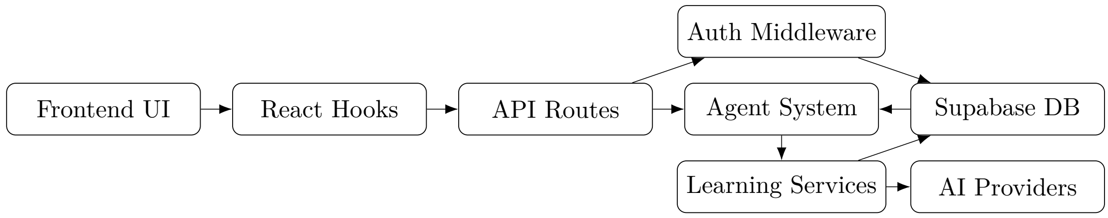

Figure 3.6: Module dependency diagram

3.11 Adaptive Strategy Selection Workflow

The adaptive strategy selector uses learner state, topic context, and feedback history to choose the most suitable learning action. The workflow replaces a simple question-to-response pipeline with a detailed agentic flow:

```
User Query -> Context Builder -> Strategy Selector -> Content Generator
-> Simulation/Visualization -> Validation -> Storage -> Feedback
```

```
Input: user query, conversation history, profile, document chunks, feedback
1. Build context from current query, profile, history, and retrieved documents.
2. Classify intent as normal chat, deep learning, quiz, flashcard, simulation,
   visualization, video, or voice.
3. Estimate learner state using confidence, engagement, failures, topic status, and
   response time.
4. Select strategy: concise explanation, guided steps, visual explanation, quiz,
   simulation, remediation, or generated video.
5. Generate the required learning artifact using the selected agent.
6. Validate output for safety, structure, and rendering support.
7. Store message, resource, selected strategy, and metadata.
8. Collect feedback and update future strategy context.
```

3.12 Security Design

The security design protects both user data and generated content:

- **Authentication:** Supabase sessions identify signed-in users before private routes are accessed.
- **Authorization:** Ownership checks ensure that users can access only their own conversations, resources, documents, and feedback.

- **Rate limiting:** Expensive routes such as chat, simulation, visualization, voice, and generation APIs are limited to control misuse and cost.
- **Safe rendering:** Generated visual artifacts are validated or rendered in controlled viewers before display.
- **Backend-only secrets:** Provider keys are stored on the backend and are never exposed to the browser.

3.13 Design Rationale

React was selected for the frontend because it supports reusable components, state-driven rendering, and a responsive single-page application structure. Vite was selected for fast development builds and simple frontend tooling. Node.js and Express were selected for the backend because they provide lightweight routing, middleware composition, streaming support, and easy integration with JavaScript-based services.

Supabase was selected because it combines authentication, PostgreSQL-backed persistence, storage, and row-level security concepts in one platform. Server-Sent Events were selected for chat streaming because they allow the browser to receive incremental text and structured events over a simple HTTP connection. This is suitable for educational responses because learners can begin reading while the response is still being generated.

The agent-based architecture was selected because different learning tasks require different specialized behavior. A single general response generator is not enough for a platform that supports explanations, RAG, quizzes, flashcards, simulations, image generation, video generation, 3D visualization, validation, and feedback adaptation.

3.14 Design Summary

The design of VisuaLearn emphasizes modularity, adaptability, reliability, and usability. The architecture separates interface, application routing, intelligence, generation modules, validation, and data persistence. The detailed workflow and diagrams show how learner queries move through context building, adaptive strategy selection, generation, validation, storage, and feedback so that the platform can support both natural conversation and structured learning.

Chapter 4

Implementation

Chapter 4

Implementation

The implementation of VisuaLearn AI converts the system design into a working full-stack learning platform. The application is divided into frontend, backend, data storage, learning content generation, adaptive learning, simulation, visualization, voice interaction, and reliability modules. This chapter explains how each major part was implemented.

4.1 Development Environment

The frontend was implemented using React with Vite as the development and build tool. Tailwind CSS was used for styling and responsive layouts. Framer Motion was used for subtle interface transitions. The backend was implemented using Node.js and Express.js. Supabase was used for authentication and database storage. The project follows a modular directory structure with separate frontend and backend folders.

4.2 Frontend Implementation

The frontend is organized using route-based pages and reusable components. The major pages include the home page, login page, signup page, new chat page, learning page, dashboard, and settings page. Shared components include the sidebar, message list, message bubble, learning workspace, voice overlay, simulation viewer, widget frame, persona cards, and feedback controls.

The learning page is the central interface for the learner. It displays generated content in organized sections such as Learn, Examples, Flashcards, Quiz, Mind Map, Simulation, and Visualization. The system avoids displaying every artifact at once on small screens. This improves readability and makes the interface usable on both desktop and mobile devices.

Custom hooks are used to manage complex frontend behavior. Chat streaming, learning content loading, simulation state, voice recording, feedback submission, authentication, progress tracking, and resource retrieval are separated into hooks. This improves maintainability and prevents large components from becoming difficult to manage.

4.3 Backend Implementation

The backend contains feature-specific route modules, middleware, services, agents, database clients, and utility functions. Middleware is used for authentication, request tracing, rate limiting, validation, and error handling. Feature modules handle chat, learning content, simulation, visualization, voice, documents, personas, user progress, and resource management.

The backend also supports streaming responses for interactive learning. Streaming improves user experience because the learner can begin reading while the response is still being prepared. All generated responses and learning resources are connected to conversations, allowing learners to return to previous sessions.

4.4 Learning Content Implementation

The learning content module generates structured educational artifacts. Instead of returning only a paragraph, the module can produce:

- Conceptual explanations
- Examples and analogies
- Flashcards
- Quiz questions
- Mind map nodes
- Summary blocks

The generated content is normalized before display. This ensures that the frontend receives predictable structures. Lazy loading is used so that resources are generated only when required. For example, flashcards can be generated when the learner opens the flashcard section rather than during the initial answer.

4.5 Adaptive Learning Implementation

The adaptive layer collects learner signals and selects a teaching strategy. The context vector includes cognitive state, engagement level, topic status, performance trend, confidence, topic difficulty, previous failures, and response time. Based on this context, the system chooses a suitable response style.

The supported teaching strategies are:

1. Visual learning for concepts needing diagrams or widgets.
2. Guided steps for procedures and algorithms.
3. Quiz mode for comprehension checking.
4. Concise explanation for quick clarification.
5. Socratic questioning for reflective learning.
6. Remediation for misconception correction.

Feedback such as quiz correctness, completion, and engagement is recorded. This creates a closed loop between learner interaction and future response selection.

4.6 Simulation Implementation

The simulation module detects whether a topic is suitable for simulation. Topics such as sorting, graph traversal, stacks, queues, heaps, linked lists, trees, state machines, grids, and timelines can be represented as state transitions. Once a simulation topic is detected, the corresponding generator creates a sequence of steps.

The frontend simulation viewer displays these steps with controls for play, pause, next, previous, and reset. This helps learners inspect intermediate states. For example, in bubble sort, the learner can see each comparison and swap. In graph traversal, the learner can see the order in which nodes are visited.

4.7 Visualization Implementation

The visualization module supports dynamic and spatial explanations. The system checks whether the device can render complex visual content. This is important because heavy visual components may not work well on all devices.

Visual widgets are displayed inside an isolated frame. This protects the rest of the application from invalid or unsafe visual output. The widget content is sanitized, and fallback messages are shown if rendering fails. This allows the system to provide rich visual learning without compromising stability.

4.8 Voice Interaction Implementation

The voice module allows learners to interact using speech. The frontend captures microphone input and connects it to a backend voice session. The backend links the voice session with the learner context and conversation history. Transcripts are synchronized with the chat so that spoken sessions remain part of the learning record.

Voice support improves accessibility and allows learners to ask quick follow-up questions naturally. It is useful during revision and for learners who prefer spoken explanations.

4.9 Persistence Implementation

Persistence is implemented using Supabase. The database stores users, conversations, messages, learning resources, personas, feedback, and adaptive decision records. Each learning resource is linked to a conversation. This allows the learner to reopen a previous session and access saved notes, flashcards, quizzes, and visual resources.

The storage design also avoids repeated generation. If a resource already exists for a conversation, the frontend can load it from storage. This improves responsiveness and reduces repeated processing.

4.10 Security and Reliability Implementation

The system includes several reliability and security mechanisms:

- Protected routes for authenticated users.
- Request validation for malformed inputs.
- Rate limiting for controlled usage.
- Safe rendering for generated visual artifacts.
- Fallback responses when a module fails.
- Structured logging for debugging and monitoring.
- Caching for repeated or expensive operations.

4.11 Implementation Summary

The implemented system provides a complete adaptive learning workflow. Learners can ask questions, receive structured explanations, access revision artifacts, run simulations, view mind maps, use voice interaction, and revisit previous sessions. The modular implementation makes the system maintainable and extensible.

Chapter 5

Testing and Results

Chapter 5

Testing and Results

Testing was performed to verify that VisuaLearn AI satisfies its functional requirements, responds correctly to learner actions, stores learning resources, and remains usable across devices. The system contains several interacting modules, so testing was carried out at the frontend, backend, database, simulation, visualization, voice, and learning workflow levels.

5.1 Testing Strategy

The testing strategy included the following:

- Functional testing of login, chat, learning resources, simulation, visualization, and settings.
- Backend route testing for valid and invalid requests.
- Manual testing of complete learner workflows.
- Simulation testing using deterministic algorithm examples.
- Responsive testing on desktop and mobile screen sizes.
- Error handling tests for missing input, unavailable resources, and rendering failures.
- Persistence testing for conversations, messages, and learning resources.

5.2 Functional Test Cases

Table 5.1: Functional Test Cases

ID	Scenario	Expected Result	Status
TC01	User login	Valid user is authenticated and redirected to the application	Passed

TC02	New conversation	A new learning session is created and shown in recent sessions	Passed
TC03	Ask question	The system returns an educational response and stores it	Passed
TC04	Open learning workspace	Learning tabs and generated resources are displayed	Passed
TC05	Generate flashcards	Flashcards appear with question and answer structure	Passed
TC06	Generate quiz	Questions, options, and feedback are shown	Passed
TC07	Generate mind map	Concepts are organized as related nodes	Passed
TC08	Detect simulation topic	Suitable procedural topics open simulation workflow	Passed
TC09	Invalid input	A controlled error or fallback response is shown	Passed
TC10	Logout	User session is cleared securely	Passed

5.3 Learning Workflow Testing

The learning workflow was tested using topics from computer science, grammar, general science, and engineering. Example queries included merge sort, breadth-first search, parts of speech, machine learning, neural networks, selection sort, and how an engine works. These topics were selected because they require different learning formats.

Algorithmic topics were suitable for simulations and guided steps. Conceptual topics were suitable for explanations, examples, flashcards, and quizzes. Grammar topics were suitable for examples and assessment. This confirmed that the system can organize learning support based on topic type.

5.4 Simulation Testing

Simulation testing focused on state correctness. For sorting algorithms, the array state was checked after each comparison and swap. For graph traversal, the visited node order was inspected. For stack and queue topics, push, pop, enqueue, and dequeue operations were checked. For tree topics, node relationships and traversal order were verified.

The results showed that the simulation module works best for deterministic processes. For topics that do not have clear state transitions, the system avoids forcing simulation and instead provides structured explanations.

5.5 Visualization Testing

Visualization testing verified whether generated visual components load safely and whether fallback messages appear when rendering is not possible. Device capability checks were used to avoid heavy visual rendering on unsupported devices. The isolated widget frame helped prevent invalid visual content from affecting the full application.

5.6 Responsive Testing

The application was tested on desktop and mobile widths. The sidebar, chat input, learning content, flashcard cards, quiz view, simulation view, and settings page were checked for overflow and readability. Mobile improvements focused on keeping the main learning content visible, reducing unnecessary side content, and preventing persona or status labels from overflowing.

5.7 Error Handling Testing

Error handling was tested by submitting incomplete requests, unavailable resources, unsuitable simulation topics, and rendering failures. The system returned controlled messages instead of exposing raw internal details. Fallback content helped preserve the learning flow even when one feature could not complete.

5.8 Observed Results

The implementation produced the following results:

- Learners can create and continue learning conversations.
- Structured learning resources are generated and displayed in separate views.
- Flashcards and quizzes support revision and self-assessment.

- Mind maps organize related concepts visually.
- Simulations support procedural understanding for algorithmic topics.
- Voice interaction supports natural question asking and accessibility.
- Persistent storage allows learners to revisit previous sessions.
- Adaptive strategy selection supports more personalized response formats.

5.9 Result Summary

Testing confirmed that VisuaLearn AI functions as an integrated adaptive learning workspace. The system performs best when topics can be represented through structured explanation, practice, visualization, or simulation. The platform successfully demonstrates the project objective of moving beyond plain answer generation toward a reusable and interactive learning environment.

Chapter 6

Conclusion and Future Enhancements

Chapter 6

Conclusion and Future Enhancements

VisuaLearn AI was developed as an adaptive multi-modal intelligent learning platform that supports structured explanations, examples, quizzes, flashcards, mind maps, simulations, visualizations, voice interaction, personalization, and persistent learning history. The project addresses the limitations of text-only learning tools by combining multiple educational artifacts into one workspace.

6.1 Conclusion

The project successfully demonstrates how a learner query can be transformed into a complete learning experience. Instead of only providing an answer, the system can organize the topic into explanation, practice, revision, visualization, and simulation. The learning workspace allows students to study the same topic from multiple angles.

The frontend implementation provides a responsive and interactive interface. The backend implementation coordinates learning content, simulation, visualization, voice, feedback, authentication, and persistence. The database layer stores conversations and learning resources so that the learner can continue studying across sessions. The adaptive layer uses learner context and interaction signals to select a suitable teaching strategy.

The completed implementation shows that educational software becomes more useful when it adapts both content and format. VisuaLearn AI therefore provides a strong foundation for personalized learning in computer science, engineering, mathematics, and related domains.

6.2 Advantages

The major advantages of the system are:

- It combines explanations, simulations, visualizations, quizzes, flashcards, and voice interaction in one platform.
- It stores learning resources so that students can revise previous sessions.
- It supports adaptive teaching strategies based on learner context.
- It improves conceptual clarity through visual and procedural learning.

- It uses a modular architecture that can be extended with new learning modules.
- It provides fallback handling and safe rendering for reliability.

6.3 Limitations

The limitations of the current system are:

- Some generated content may require teacher review for high-stakes academic use.
- Complex visualizations depend on device capability.
- Voice interaction depends on microphone access and network stability.
- Personalization becomes more effective only after sufficient learner interaction data is collected.
- Some abstract topics cannot be converted into meaningful simulations.

6.4 Future Enhancements

Future enhancements can include:

- More subject-specific simulation generators for physics, mathematics, and engineering.
- Teacher dashboard for monitoring learner progress.
- Detailed learning outcome measurement using pre-test and post-test workflows.
- Improved spaced repetition scheduling for flashcards.
- Collaborative study rooms for group learning.
- Offline export of selected learning artifacts.
- Stronger accessibility controls for different learner needs.
- Larger-scale evaluation with real learners to measure learning impact.

6.5 Final Remarks

VisuaLearn AI provides a practical implementation of an adaptive learning workspace. It combines modular software design with educational principles such as personalization, active learning, revision, and visualization. The project can be extended further into a robust learning assistant for academic and self-learning environments.

Appendix A

Appendix

Appendix A

Appendix

A.1 Major Modules

The major implementation modules of VisuaLearn AI are:

- Frontend learning workspace
- Chat and conversation module
- Learning content module
- Flashcard and quiz module
- Mind map module
- Simulation module
- Visualization and widget rendering module
- Voice interaction module
- Persona and personalization module
- Feedback and adaptive learning module

A.2 Sample Learning Flow

A typical learning flow is:

1. Learner asks a question such as “Explain merge sort”.
2. The system identifies that the topic is procedural.
3. A guided explanation is generated.
4. The learner opens the simulation tab.
5. The simulation shows divide and merge steps.
6. Flashcards and quiz questions are generated for revision.
7. Learner feedback is stored for future personalization.

A.3 Glossary

Learning Artifact
A generated resource such as notes, quiz, flashcards, mind map, simulation, or visualization.

Adaptive Strategy
The selected teaching format for a learner response.

Simulation A step-by-step representation of a procedural topic.

Mind Map A visual representation of relationships between concepts.

Persistence Storage of conversations and generated resources for future access.

BIBLIOGRAPHY

[1] F. Martin and D. U. Bolliger, "Engagement matters: Student perceptions on the importance of engagement strategies in the online learning environment," *Online Learning*, vol. 22, no. 1, pp. 205–222, 2018. DOI: 10.24059/olj.v22i1.1092.

[2] Instructure, "2023 state of student success and engagement in higher education," Instructure, 2023.

[3] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge University Press, 2009, ISBN: 9780521735353.

[4] O. Chernikova, N. Heitzmann, M. Stadler, D. Holzberger, T. Seidel, and F. Fischer, "Simulation-based learning in higher education: A meta-analysis," *Review of Educational Research*, vol. 90, no. 4, pp. 499–541, 2020. DOI: 10.3102/0034654320933544.

[5] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988. DOI: 10.1207/s15516709cog1202_4.

[6] B. S. Bloom, "The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring," *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984. DOI: 10.3102/0013189X013006004.

[7] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011. DOI: 10.1080/00461520.2011.611369.

[8] W. Ma, O. O. Adesope, J. C. Nesbit, and Q. Liu, "Intelligent tutoring systems and learning outcomes: A meta-analysis," *Journal of Educational Psychology*, vol. 106, no. 4, pp. 901–918, 2014. DOI: 10.1037/a0037123.

[9] E. Kasneci, K. Sessler, S. Küchemann, *et al.*, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102 274, 2023. DOI: 10.1016/j.lindif.2023.102274.

[10] L. Chen, P. Chen, and Z. Lin, "Artificial intelligence in education: A review," *IEEE Access*, vol. 8, pp. 75 264–75 278, 2020. DOI: 10.1109/ACCESS.2020.2988510.

[11] P. Brusilovsky and E. Millan, "User models for adaptive hypermedia and adaptive educational systems," in *The Adaptive Web*, Springer, 2007, pp. 3–53. DOI: 10.1007/978-3-540-72079-9_1.